

Volume 02

WarrantyGraph

Algorithms & Theory

FMEA · Bayesian diagnosis · TF-IDF hybrid · GraphRAG · parts scoring

Full project library

This multi-volume set covers the entire repository — not a single session. Algorithms, theory, annotated code, RDF/OWL, pipelines, and deploy are all included.

1. FMEA / FMECA (reliability engineering)

What: Failure Mode and Effects Analysis rates each failure mode on ordinal scales Severity (S), Occurrence (O), Detection (D). Classic $RPN = S \times O \times D$. Standards: MIL-STD-1629A, SAE J1739, AIAG-VDA FMEA Handbook 2019.

Why: Industry-defensible triage instead of opaque ML scores. AIAG-VDA prefers **Action Priority (H/M/L)** because multiplying ordinals can reverse ranks (Kmenta & Ishii, 2004).

How we ground S / O / D in the graph

Factor	Graph source	Mapping idea
Severity S	Worst Symptom.severity on INDICATES path	critical→10, high→8, medium→5, low→2
Occurrence O	Count Claim HistoricalResolution -CONFIRMED→ FM	More field evidence → higher O
Detection D	Count DiagnosticStep -CONFIRMS→ FM (+ error codes)	More coverage → easier detect → lower D
RPN	$S \times O \times D$	Continuity with classic FMEA
Action Priority	Severity-dominated rules	High/Medium/Low triage

Formulas

$$RPN = S \times O \times D \quad (\text{range } 1 \dots 1000)$$

Code

Implemented in graph/reliability.py: severity_rating, occurrence_rating, detection_rating, rpn, action_priority.

2. Naive-Bayes diagnostic inference

What: Given observed symptoms S, rank failure modes by posterior probability. References: Pearl (1988); Russell & Norvig AIMA.

$$P(\text{fm} \mid S) \propto P(\text{fm}) \times \prod_i P(s_i \mid \text{fm})$$

Then **normalize** across candidate failure modes so posteriors sum to 1. This is a proper probability distribution over the differential diagnosis.

Term	Meaning in our system
$P(\text{fm})$ prior	From FMEA Occurrence rating → occurrence_prior(O) = O/10 (relative only)
$P(s \text{fm})$ likelihood	Edge (Symptom)-[:INDICATES {confidence}]->(FailureMode)
Missing edge	DEFAULT_MISS_LIKELIHOOD = 0.15 (not zero — avoids wipeout)
Observed symptoms	Strong matches ≥ symptom_match_min_score (not weak secondary)

Worked mini-example (whiteboard)

```
Symptoms observed: s1, s2
FM_A: prior=0.6, P(s1|A)=0.9, P(s2|A)=0.8 → score = 0.6*0.9*0.8 = 0.432
FM_B: prior=0.4, P(s1|B)=0.3, P(s2|B)=0.5 → score = 0.4*0.3*0.5 = 0.060
Normalize: P(A|S)=0.432/0.492=0.88, P(B|S)=0.12 → rank A first
```

Why not pure neural ranking?

Warranty needs auditability and reproducibility. Same graph + symptoms ⇒ same posterior every time (unit-tested in tests/test_reliability.py).

3. Dominance boost & recommendation strength

A posterior of 0.57 vs second 0.33 is a clearer winner than 0.57 vs 0.55. dominance_boost gently raises overall confidence when top/second ratio ≥ 1.8 and top ≥ 0.55, capped (default 0.92) so we never claim false certainty.

recommendation_strength maps (posterior, graph edge strength, language match, dominance) to Strong / Moderate / Weak / Insufficient data — categorical triage like Action Priority.

4. Hybrid symptom retrieval (lexical + TF-IDF)

Problem: Users paraphrase catalog symptoms. Pure token overlap is brittle; full embeddings need extra infra.

TF-IDF: term frequency × inverse document frequency. Rare terms weigh more. Cosine similarity between query vector and symptom description vector.

$$\text{sim}(q, d) = (q \cdot d) / (\|q\| \|d\|) \quad \text{in TF-IDF space}$$

Hybrid: blended = 0.45·lexical + 0.55·tfidf; if both ≥ 0.3 take max (agreement boost). Threshold filter; backfill weak secondary matches for UI context only.

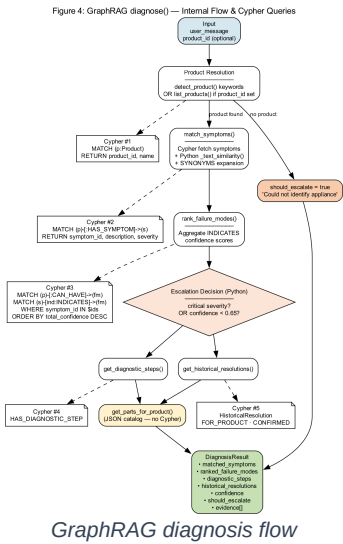
Code

graph/symptom_retrieval.py · used by graph_rag.match_symptoms when use_hybrid_symptom_matching=true.

5. GraphRAG algorithm (runtime path)

GraphRAG here means: **retrieve multi-hop evidence from a knowledge graph**, then assemble an answer. It is not “chunk PDFs into a vector store” (though vectors could be added later).

```
diagnose(message, product_id?, asset_id?):
1. resolve_product_for_diagnosis # message keywords / CRM asset / explicit id
2. match_error_codes + match_symptoms (hybrid scores)
3. strong_symptom_ids = score ≥ min_score # only these enter Bayes
4. rank_failure_modes via Cypher INDICATES + FMEA/Bayes
5. composite confidence + dominance + recommendation_strength
6. diagnostic_engine tree for top FM
7. parts_predictor(fm_posterior)
8. escalation_rules (critical / low conf / ambiguity margin)
9. format_diagnosis_response + provenance + subgraph
```



[diagram 38-reliability-diagnosis-engine.png not rendered]

6. Parts prediction scoring

$$\text{prediction_score} \approx P(\text{fm}|\text{symptoms}) \times \text{source_reliability} \times P(\text{part}|\text{fm})$$

Source path	Reliability weight	Meaning
REQUIRES_PART edge	1.00	Engineering primary part link
SKU_FIT	0.90	Compatible with asset SKU
CLAIM_PRECEDENT	0.85	Used on past confirmed claims
BOM_COMPONENT	0.70	Via IMPACTS_COMPONENT → REALIZED_BY

Why multiply by FM posterior?

A part for an unlikely failure mode must not outrank a part for the leading diagnosis. Parts never appear more likely than the FM they treat.

7. Diagnostic decision trees

Graph edges NEXT_STEP (with optional conditions), CONFIRMS, RULES_OUT form a troubleshooting automaton. Traversal is deterministic graph walk — not LLM planning. Ambiguity for escalation uses posterior margin between top-2 FMs (settings.diagnosis_ambiguity_margin).

8. Escalation decision logic (theory of when not to auto-resolve)

- Critical severity symptom → always human
- Low overall confidence AND not strong-graph → escalate
- Weak language match → escalate (wrong catalog mapping risk)
- Ambiguous posteriors (margin < threshold) → escalate
- Strong graph evidence can avoid escalate even if language is imperfect

9. Systems algorithms (runtime scale)

- **Sliding-window rate limit:** count events in last W seconds; Redis ZSET + Lua for multi-pod
- **TTL cache:** store ontology/product subgraphs; invalidate on ETL load
- **Admission control:** semaphore / Redis INCR for max concurrent diagnoses
- **Parallel map:** thread pool for independent I/O (connectors); serial transform for determinism

10. Theory quick reference card

Topic	One-liner	Module
FMEA	S,O,D → RPN + Action Priority	reliability.py
Bayes	$P(fm S) \propto P(fm) \prod P(s fm)$, normalize	reliability.bayesian_posteriors
TF-IDF hybrid	lexical + cosine TF-IDF blend	symptom_retrieval.py
GraphRAG	Cypher multi-hop + assemble answer	graph_rag.diagnose
Parts	posterior × reliability × edge P	parts_predictor.py
Dominance	boost clear winners carefully	dominance_boost

11. Algorithms as What/Where/When/How/Why

WHAT	FMEA S/O/D + RPN + Action Priority
WHERE	graph/reliability.py; wired from graph_rag._apply_fmea_and_posteriors
WHEN	During failure-mode ranking for each diagnose
HOW	S=worst symptom severity; O=claim/resolution count; D=step/error coverage; RPN=S*O*D; AP rules
WHY	Industry-defensible risk triage; Severity-led AP avoids pure RPN rank reversals
WHAT	Naive-Bayes posterior over failure modes
WHERE	reliability.bayesian_posteriors
WHEN	After strong symptoms are selected ($\geq \text{min_score}$)
HOW	$\text{score} = P(fm) \times \prod P(s fm)$; missing edge uses miss=0.15; normalize sum to 1
WHY	Differential diagnosis as probability; deterministic & unit-tested
WHAT	Hybrid lexical + TF-IDF symptom match
WHERE	symptom_retrieval.hybrid_symptom_score · graph_rag.match_symptoms
WHEN	Each diagnose after product is known
HOW	Token overlap + TF-IDF cosine; blend weights; agreement boost
WHY	Paraphrases without embedding infrastructure
WHAT	Parts multi-source score
WHERE	parts_predictor.predict_parts
WHEN	After top FM chosen
HOW	posterior × path reliability × edge probability; merge by part_id
WHY	Part never outranks its failure mode; BOM + claims evidence